Analytic Derivation of Singularities in Cubic Bézier Curves *

Takeshi Bashomatsu †

October 31, 2026

Abstract

Cubic Bézier curves can exhibit singularities such as self-intersections, cusps, and inflection points depending on the arrangement of control points. Although basic properties of Bézier curves are well documented and widely used in practice [1], explicit formulations for singular points are not always readily found in classical references. In this paper, we derive analytic expressions for this. We also provide a unified table for distinguishing each type of singularity and present a C implementation example to facilitate practical use.

1 Background and Related Work

Cubic Bézier curves are widely used to model smooth curves, but in industrial applications, the presence of self-intersections or cusps can have a significant impact, and it is necessary to guarantee that such undesirable features are not present in input data. In many cases, the input data contains a large number of Bézier curves, requiring a lightweight method for rapid processing.

Computing the singularities of a cubic Bézier curve involves solving polynomials derived from the curve's parametric functions and curvature. Stone and DeRose showed that by classifying singularities and transforming Bézier curves into canonical forms, each type can be mapped in the XY plane [2]. However, converting to canonical form has disadvantages, such as increased computation and issues handling inflection points near $t = 0$, which may require preprocessing to reverse the control point order. 白山 presented analytic expressions for directly computing self-intersection points from control point coordinates [3]. This paper improves on that to reduce numerical errors and derives expressions for inflection points. We then unify the discrimination and derivation of singularities.

*This article has been reconstructed based on content previously published in the following post:
Qiita - "Analytic Derivation of Singularities in Cubic Bézier Curves"
https://qiita.com/kumanomi1000/items/7621db2d05b87884d49d

†bashomatsutakeshi@gmail.com

2 Preliminaries

Let the coordinates of an arbitrary point on a cubic Bézier curve be:

$$x = f(t) = a_0t^3 + a_1t^2 + a_2t + a_3$$
$$y = g(t) = b_0t^3 + b_1t^2 + b_2t + b_3$$

where $t \in [0, 1]$ and according to the definition of a cubic Bézier curve [4], the coefficients a_i and b_i are defined in terms of control points $\boldsymbol{B}_0, \boldsymbol{B}_1, \boldsymbol{B}_2, \boldsymbol{B}_3$:

$$\begin{aligned}
a_0 &= -B_{0x} + 3B_{1x} - 3B_{2x} + B_{3x} \\
a_1 &= 3B_{0x} - 6B_{1x} + 3B_{2x} \\
a_2 &= -3B_{0x} + 3B_{1x} \\
a_3 &= B_{0x} \\
b_0 &= -B_{0y} + 3B_{1y} - 3B_{2y} + B_{3y} \\
b_1 &= 3B_{0y} - 6B_{1y} + 3B_{2y} \\
b_2 &= -3B_{0y} + 3B_{1y} \\
b_3 &= B_{0y}
\end{aligned}$$

Also define:

$$\begin{aligned}
v &= a_0b_1 - a_1b_0 \\
w &= a_0b_2 - a_2b_0 \\
u &= a_1b_2 - a_2b_1
\end{aligned}$$

3 Self-Intersection Points and Cusp Point

A self-intersection occurs when there exist two distinct parameters t_1 and t_2 such that $f(t_1) = f(t_2)$ and $g(t_1) = g(t_2)$. For x coordinates:

$$\begin{aligned}
&a_0t_1^3 + a_1t_1^2 + a_2t_1 + a_3 = a_0t_2^3 + a_1t_2^2 + a_2t_2 + a_3 \\
&a_0(t_1^3 - t_2^3) + a_1(t_1^2 - t_2^2) + a_2(t_1 - t_2) = 0
\end{aligned}$$

Dividing both sides by $(t_1 - t_2)$:

$$a_0(t_1^2 + t_1t_2 + t_2^2) + a_1(t_1 + t_2) + a_2 = 0 \tag{1}$$

Similarly for y :

$$b_0(t_1^2 + t_1t_2 + t_2^2) + b_1(t_1 + t_2) + b_2 = 0 \tag{2}$$

Transforming (1) and substituting it into (2), and rearranging for t_1, we get:

$$\begin{aligned}
&\frac{b_0}{a_0}(-a_1(t_1 + t_2) - a_2) + b_1(t_1 + t_2) + b_2 = 0 \\
&t_1 = -t_2 - \frac{b_0a_2 - b_2a_0}{b_0a_1 - b_1a_0} \\
&t_1 = -t_2 - \frac{w}{v}
\end{aligned}$$

Substituting it into (1) :

$$a_0 t_2^2 + \frac{a_0 w t_2}{v} + \frac{a_0 w^2}{v^2} - \frac{a_1 w}{v} + a_2 = 0$$

Multiplying both sides by v/a_0 gives:

$$v t_2^2 + w t_2 + \frac{w^2}{v} - \frac{a_1 w}{a_0} + \frac{a_2 v}{a_0} = 0 \tag{3}$$

When $v = 0$, there are no self-intersection or cusp. When $v \neq 0$, by applying the quadratic formula and using the symmetry relationship between t_1 and t_2, we get:

$$t_1, t_2 = \frac{-w \pm \sqrt{-3w^2 + 4vu}}{2v}$$

If both t_1 and t_2 lie within $[0, 1]$, a self-intersection exists. When $-3w^2 + 4vu = 0$, we have $t_1 = t_2$ which corresponds to a cusp.

4 Inflection Points

The curvature of a parametric curve [5] is:

$$\kappa = \frac{x'y'' - y'x''}{\left(x'^2 + y'^2\right)^{3/2}},$$

and inflection points occur when the numerator $\kappa_a = x'y'' - y'x''$ is zero. Substituting derivatives of x and y:

$$\begin{aligned} x' &= 3a_0 t^2 + 2a_1 t + a_2 \\ x'' &= 6a_0 t + 2a_1 \\ y' &= 3b_0 t^2 + 2b_1 t + b_2 \\ y'' &= 6b_0 t + 2b_1 \end{aligned}$$

into κ_a and simplifying yields:

$$\begin{aligned} &\kappa_a = 3(a_1 b_0 - a_0 b_1)t^2 + 3(a_2 b_0 - a_0 b_2)t + a_2 b_1 - a_1 b_2 = 0 \\ &3vt^2 + 3wt + u = 0 \end{aligned} \tag{4}$$

By applying the quadratic formula, we get:

$$t = \frac{-3w \pm \sqrt{3(3w^2 - 4uv)}}{6v}$$

With special case when $v = 0 \wedge w \neq 0$, (4) becomes a linear equation, we get:

$$t = -\frac{u}{w}$$

If the resulting t falls within $[0, 1]$, it is an inflection point. Note that there is no inflection points when $v = 0 \wedge w = 0$.

5 Unified Discrimination

All derived expressions include a common discriminant $D = 4uv - 3w^2$. Using D, singularities can be classified and derived:

<table>
<tr><td rowspan="2">$D < 0$</td><td rowspan="2">Inflection points</td><td>$v \neq 0$</td><td>$t = \dfrac{-w \pm \sqrt{-\frac{D}{3}}}{2v}$</td></tr>
<tr><td>$v = 0 \land w \neq 0$</td><td>$t = -\dfrac{u}{3w}$</td></tr>
<tr><td>$D = 0 \land v \neq 0$</td><td colspan="2">Cusp</td><td>$t = -\dfrac{w}{2v}$</td></tr>
<tr><td>$D > 0 \land v \neq 0$</td><td colspan="2">Self-intersection</td><td>$t = \dfrac{-w \pm \sqrt{D}}{2v}$</td></tr>
</table>

6 Discussion

In high-order polynomials, floating-point errors can significantly affect results. Normalizing control point coordinates can help mitigate these errors. The implementation example included normalizes all coordinates to the range $[0, 1]$ before computation.

7 Conclusion

This paper presents analytic expressions for self-intersection points, cusps, and inflection points for cubic Bézier curves, along with a unified table for classification. A C language implementation example is provided in the appendix.

References

[1] P. Bézier. *Mathematical and Practical Possibilities of UNISURF*. Computer Aided Geometric Design - Proceedings of a Conference Held at The University of Utah, Salt Lake City, Utah, March 18-21, 1974, pp.127-152. ISBN 0-12-079050-5.

[2] Maureen C. Stone, Tony D. DeRose. *A Geometric Characterization of Parametric Cubic Curves*. ACM Transactions on Graphics, Vol. 8, No. 3, July 1989, pp.147-163.

[3] Zenn 白山風露『自己交差する 3 次ベジェ曲線の交差点を求める』
https://zenn.dev/kazatsuyu/scraps/a362d9d1c35da6

[4] Gerald Farin. *Curves and Surfaces for Computer Aided Geometric Design: A Practical Guide*. Academic Press, ISBN 0-12-249050-9, p.42, 1988.

[5] Wikipedia – Curvature
https://en.wikipedia.org/wiki/Curvature

Appendix A Implementation Example

Listing 1: bzsingular.cpp

```
#include <cmath>
#include <QPointF>
#include <QList>
using namespace std;

#define EZERO 1e-14 // Values closer to zero than this are treated as zero

#define PW3(x) ((x)*(x)*(x))
#define PW2(x) ((x)*(x))

// Compute the coefficients of a cubic Bezier curve
void coefBezier(
  QList<QPointF> &cp, // IN Bezier curve control points
  double *a, // OUT x-coordinate coefficients
  double *b) // OUT y-coordinate coefficients
{
   a[0]=-cp[0].x()+3*cp[1].x()-3*cp[2].x()+cp[3].x();
   a[1]=3*cp[0].x()-6*cp[1].x()+3*cp[2].x();
   a[2]=-3*cp[0].x()+3*cp[1].x();
   a[3]=cp[0].x();
   b[0]=-cp[0].y()+3*cp[1].y()-3*cp[2].y()+cp[3].y();
   b[1]=3*cp[0].y()-6*cp[1].y()+3*cp[2].y();
   b[2]=-3*cp[0].y()+3*cp[1].y();
   b[3]=cp[0].y();
}

// Compute a point on a cubic parametric curve
QPointF pointBezier(
  double *a, // IN x-coordinate coefficients
  double *b, // IN y-coordinate coefficients
  double t) // IN Parameter
{
   return QPointF(a[0]*PW3(t)+a[1]*PW2(t)+a[2]*t+a[3],
               b[0]*PW3(t)+b[1]*PW2(t)+b[2]*t+b[3]);
}

// Normalize control point coordinates to the range [0, 1]
void normCP(
  QList<QPointF> &cp, // IN Control points of curve
  QList<QPointF> &ncp) // OUT Normalized control points of curve
{
   double xmin=cp[0].x();
   double xmax=cp[0].x();
   double ymin=cp[0].y();
   double ymax=cp[0].y();
   foreach(QPointF p,cp.mid(1)){
      if(xmin>p.x()) xmin=p.x();
      if(xmax<p.x()) xmax=p.x();
```

```
      if(ymin>p.y()) ymin=p.y();
      if(ymax<p.y()) ymax=p.y();
   }
   double m=max(xmax-xmin,ymax-ymin);
   ncp.clear();
   foreach(QPointF p,cp)
      ncp<<QPointF((p.x()-xmin)/m,(p.y()-ymin)/m);
}

// Compute singular points of cubic Bezier curve
bool vBezierSingular( // RETURN true: success
  QList<QPointF> &cp, // IN Control points of cubic Bezier curve (4 points)
  QList<QPointF> &opI, // OUT Inflection points (0 to 2 points)
  QList<QPointF> &opC, // OUT Cusp point (0 or 1 point)
  QList<QPointF> &opS) // OUT Self-intersection point (0 or 1 point)
{
   opI.clear();
   opC.clear();
   opS.clear();
   if(cp.size()!=4)
      return false;

   // Compute coefficients of the Bezier curve
   double a[4],b[4];
   coefBezier(cp,a,b);

   // Compute normalized coordinates of control points and coefficients
   QList<QPointF> ncp;
   normCP(cp, ncp);
   double na[4],nb[4];
   coefBezier(ncp,na,nb);

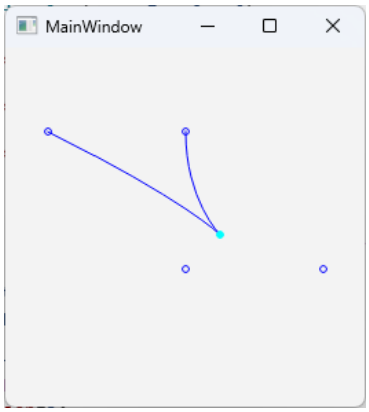
   // Compute singular points
   double v=na[0]*nb[1]-na[1]*nb[0];
   double w=na[0]*nb[2]-na[2]*nb[0];
   double u=na[1]*nb[2]-na[2]*nb[1];
   double D=4*u*v-3*PW2(w);
   if(abs(v)>=EZERO){
      if(D<=-EZERO){
         // Inflection points
         double sqd=sqrt(-D/3);
         double t1=(-w+sqd)/2/v;
         double t2=(-w-sqd)/2/v;
         if(t1>=0&&t1<=1)
            opI<<pointBezier(a,b,t1);
         if(t2>=0&&t2<=1)
            opI<<pointBezier(a,b,t2);
      }else if(D<EZERO){
         // Cusp
         double t=-w/2/v;
         if(t>=0&&t<=1)
```

```
            opC<<pointBezier(a,b,t);
      }else{
         // Self-intersection
         double sqd=sqrt(D);
         double t1=(-w+sqd)/2/v;
         double t2=(-w-sqd)/2/v;
         if(t1>=0&&t1<=1&&t2>=0&&t2<=1)
            opS<<pointBezier(a,b,t1);
      }
   }else{
      if(D<=-EZERO&&abs(w)>=EZERO){
         // Inflection point
         double t=-u/3/w;
         if(t>=0&&t<=1)
            opI<<pointBezier(a,b,t);
      }
   }
   return true;
}
```

Appendix B Qt Sample Application

When you start the program, a cubic Bézier curve and a cusp point (cyan) will be displayed. Dragging and moving the control points will update the curve and detect singular points. A self-intersection point (red) and inflection points (green) will be displayed when they are existing.

Example of display:



Listing 2: mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QGraphicsScene>
#include <QGraphicsItem>
#include <QGraphicsSceneMouseEvent>
```

```
QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class Scene : public QGraphicsScene
{
   Q_OBJECT
public:
   explicit Scene(QObject *parent = nullptr){ Q_UNUSED(parent) }

signals:
   void mouseMove(QGraphicsSceneMouseEvent *mouseEvent);
   void mousePress(QGraphicsSceneMouseEvent *mouseEvent);

protected:
   virtual void mouseMoveEvent(QGraphicsSceneMouseEvent *mouseEvent)
      override
   { emit mouseMove(mouseEvent); }
   virtual void mousePressEvent(QGraphicsSceneMouseEvent *mouseEvent)
      override
   { emit mousePress(mouseEvent); }
};

class MainWindow : public QMainWindow
{
   Q_OBJECT

public:
   MainWindow(QWidget *parent = nullptr);
   ~MainWindow();

private slots:
   void mouseMove(QGraphicsSceneMouseEvent *mouseEvent);
   void mousePress(QGraphicsSceneMouseEvent *mouseEvent);

private:
   Scene scene;
   QGraphicsEllipseItem *rc[4];
   QGraphicsPathItem *ci;
   QList<QGraphicsEllipseItem *> ri;
   QList<QPointF> cp;
   int scp;

private:
   Ui::MainWindow *ui;
};
#endif // MAINWINDOW_H
```

Listing 3: mainwindow.cpp

```
#include "mainwindow.h"
```

```
#include "ui_mainwindow.h"

bool vBezierSingular(
   QList<QPointF> &cp, // IN Control points of cubic Bezier curve (4 points)
   QList<QPointF> &opI, // OUT Inflection points (0 to 2 points)
   QList<QPointF> &opC, // OUT Cusp point (0 or 1 point)
   QList<QPointF> &opS); // OUT Self-intersection point (0 or 1 point)

MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent)
, ui(new Ui::MainWindow)
{
   ui->setupUi(this);
   scene.setSceneRect(0,0,ui->graphicsView->width()-2,ui->graphicsView->
      height()-2);
   ui->graphicsView->setScene(&scene);
   ui->graphicsView->setRenderHint(QPainter::Antialiasing);
   // example of cusp
   cp<<QPointF(120,50);
   cp<<QPointF(120,150);
   cp<<QPointF(220,150);
   cp<<QPointF(20,50);
   for(int i=0;i<4;i++){
      rc[i]=scene.addEllipse(cp[i].x()-2.5,cp[i].y()-2.5,5,5,QPen(Qt::blue))
         ;
   }
   {
      QPainterPath curve(cp[0]);
      curve.cubicTo(cp[1],cp[2],cp[3]);
      ci=scene.addPath(curve,QPen(QBrush(Qt::blue),1));
   }
   QList<QPointF> opI,opC,opS;
   vBezierSingular(cp,opI,opC,opS);
   foreach (QPointF p,opI)
      ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::green),QBrush(Qt
         ::green));
   foreach (QPointF p,opC)
      ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::cyan),QBrush(Qt
         ::cyan));
   foreach (QPointF p,opS)
      ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::red),QBrush(Qt::
         red));
   scp=-1;

   connect(&scene,SIGNAL(mouseMove(QGraphicsSceneMouseEvent*)),this,SLOT(
      mouseMove(QGraphicsSceneMouseEvent*)));
   connect(&scene,SIGNAL(mousePress(QGraphicsSceneMouseEvent*)),this,SLOT(
      mousePress(QGraphicsSceneMouseEvent*)));
}

MainWindow::~MainWindow()
```

```
{
   delete ui;
}

void MainWindow::mouseMove(QGraphicsSceneMouseEvent *mouseEvent)
{
   if((mouseEvent->buttons()&Qt::LeftButton)&&scp>=0){
      QPointF pos=mouseEvent->scenePos();
      cp[scp]=pos;
      rc[scp]->setRect(pos.x()-2.5,pos.y()-2.5,5,5);
      QPainterPath curve(cp[0]);
      curve.cubicTo(cp[1],cp[2],cp[3]);
      ci->setPath(curve);
      foreach(QGraphicsEllipseItem *gi,ri){
         scene.removeItem(gi);
         delete gi;
      }
      ri.clear();
      QList<QPointF> opI,opC,opS;
      vBezierSingular(cp,opI,opC,opS);
      foreach (QPointF p,opI)
         ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::green),QBrush
            (Qt::green));
      foreach (QPointF p,opC)
         ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::cyan),QBrush(
            Qt::cyan));
      foreach (QPointF p,opS)
         ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::red),QBrush(
            Qt::red));
   }
}

void MainWindow::mousePress(QGraphicsSceneMouseEvent *mouseEvent)
{
   if(mouseEvent->buttons() & Qt::LeftButton){
      QPointF pos = mouseEvent->scenePos();
      scp=-1;
      for(int i=0;i<4;i++){
         if(QLineF(pos,rc[i]->rect().center()).length()<=5)
            scp=i;
      }
   }
}
```

Listing 4: mainwindow.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>MainWindow</class>
 <widget class="QMainWindow" name="MainWindow">
  <property name="geometry">
   <rect>
```

```
   <x>0</x>
   <y>0</y>
   <width>261</width>
   <height>261</height>
  </rect>
 </property>
 <property name="windowTitle">
  <string>MainWindow</string>
 </property>
 <widget class="QWidget" name="centralwidget">
  <widget class="QGraphicsView" name="graphicsView">
   <property name="geometry">
    <rect>
     <x>10</x>
     <y>10</y>
     <width>241</width>
     <height>241</height>
    </rect>
   </property>
   <property name="minimumSize">
    <size>
     <width>100</width>
     <height>100</height>
    </size>
   </property>
   <property name="mouseTracking">
    <bool>true</bool>
   </property>
   <property name="alignment">
    <set>Qt::AlignmentFlag::AlignLeading|Qt::AlignmentFlag::AlignLeft|Qt::
        AlignmentFlag::AlignTop</set>
   </property>
  </widget>
 </widget>
</widget>
<resources/>
<connections/>
</ui>
```